

Informe Técnico de Arquitectura: Plataforma VeloUrbe

1. Introducción

El presente informe técnico detalla la arquitectura, el diseño estructurado y los lineamientos de desarrollo aplicados en la construcción de **VeloUrbe**, una plataforma orientada a la gestión de arriendo de patinetas eléctricas (scooters) en entornos urbanos. El objetivo de este documento es proporcionar una visión clara de la solución, evidenciando la aplicación de patrones de diseño modernos, prácticas de Clean Code y un enfoque riguroso en el desarrollo de software empresarial.

La plataforma está compuesta por **10 microservicios funcionales** coordinados por un **API Gateway** y un **BFF (Backend for Frontend)**, siguiendo el flujo en capas **Gateway** → **BFF** → **microservicios**. Ha sido construida garantizando alta disponibilidad, escalabilidad y un bajo acoplamiento mediante el uso de contenedores y tecnologías base como Spring Boot 3.5 y Java 21, estableciendo una infraestructura sólida para soportar operaciones de movilidad inteligente.

2. Desarrollo

2.1. Contexto, Requerimientos y Casos de Uso

Contexto: VeloUrbe resuelve el desafío de la movilidad urbana de última milla ofreciendo un ecosistema digital que permite a los ciudadanos arrendar patinetas eléctricas de forma ágil y segura. La adopción de una arquitectura de microservicios asegura que la gestión de identidades, el inventario de patinetas, los pagos, la logística y la orquestación de la experiencia del cliente final operen de manera desacoplada y resiliente.

Requerimientos Funcionales (RF)

ID	Descripción
RF01	Gestión de Usuarios: Registro e inicio de sesión emitiendo tokens JWT firmados (HS256) que integran los roles y permisos del usuario.
RF02	Inventario de Patinetas: Operaciones CRUD completas sobre los scooters, incluyendo monitoreo de batería, ubicación geográfica y disponibilidad.
RF03	Gestión de Arriendos: Lógica transaccional para iniciar y finalizar viajes, registrando marcas de tiempo precisas y calculando duraciones totales.
RF04	Pagos: Procesamiento del ciclo de vida completo del pago de un arriendo (procesar, completar, cancelar, reembolsar).
RF05	Notificaciones y Analítica: Envío de notificaciones EMAIL/SMS con historial por usuario, y registro de eventos de arriendo con estadísticas por usuario y de sistema.
RF06	Logística, Mantención y Soporte: Ubicación GPS de la flota con búsqueda por radio, ciclo de vida de issues de mantención y tickets de soporte al cliente.
RF07	Estaciones: Gestión de estaciones de carga/estacionamiento con capacidad, ocupación (dock/undock) y búsqueda por cercanía geográfica (Haversine).
RF08	Reseñas: Calificación de arriendos (1–5, una reseña por arriendo) con promedio agregado por patineta.
RF09	BFF: Punto único de acceso a los microservicios — proxy de todas las rutas /api/** y vistas agregadas (dashboard, resumen de arriendos) en una única petición de red.

Requerimientos No Funcionales (RNF)

- **Seguridad:** Autenticación *stateless* validada mediante JwtAuthFilter en cada microservicio de forma descentralizada; encriptación de contraseñas mediante el algoritmo BCrypt.
- **Arquitectura e Infraestructura:** Despliegue orquestado en contenedores Docker con 13 contenedores (Gateway, BFF, 10 microservicios y PostgreSQL), garantizando bases de datos independientes por dominio y

enrutamiento centralizado en capas.

- **Trazabilidad:** Logging estructurado en formato JSON mediante logstash-logback-encoder para facilitar la indexación y el monitoreo centralizado.
- **Calidad de Código:** Patrón CSR (Controller → Service → Repository) en todos los microservicios, DTOs inmutables con Java Records y respuestas RESTful con convenciones HATEOAS.

Casos de Uso Principales

- 1 **Registro y Autenticación:** El cliente envía sus credenciales al Gateway, que las enruta a través del BFF hacia el servicio de autenticación, donde se procesan, hashen y persisten de forma segura, retornando un JWT que habilita el acceso a los módulos protegidos del ecosistema.
- 2 **Ciclo de Vida del Arriendo:** Un usuario autenticado solicita el inicio de un viaje. El sistema verifica la disponibilidad transaccional en el inventario, muta el estado del scooter a "en uso" e inicializa el registro del arriendo. Al finalizar, el sistema libera el recurso, contabiliza el tiempo, procesa el pago y permite calificar la experiencia con una reseña.
- 3 **Orquestación de Dashboard:** Al renderizar la interfaz principal, el cliente consulta al BFF, que ejecuta llamadas concurrentes no bloqueantes vía WebClient hacia los servicios subyacentes, ensamblando un payload unificado y optimizado.

2.2. Arquitectura de Microservicios

La plataforma distribuye su lógica de dominio en componentes delimitados (*Bounded Contexts*) organizados en un flujo de tres capas: el **API Gateway va primero** como punto de entrada único, **seguido del BFF**, y el **BFF apunta a todos los microservicios**. El Gateway no contiene código: es exclusivamente configuración de enrutamiento.

Servicio	Puerto	Responsabilidad	Base de datos
api-gateway	8080	Punto de entrada único, CORS y enrutamiento de /api/** al BFF (sin código). Aloja el Swagger centralizado.	—
bff-service	8083	Proxy hacia los 10 microservicios + vistas agregadas para el frontend (/api/bff/**).	Sin persistencia
user-auth-service	8081	Registro, login, emisión de JWT y gestión de usuarios.	db_scooter_users
scooter-rental-service	8082	Inventario de patinetas y lógica ACID de arriendos.	db_scooter_rentals
payment-service	8084	Pagos de arriendos: procesar, completar, cancelar, reembolsar.	db_scooter_payments
notification-service	8085	Notificaciones EMAIL/SMS e historial por usuario.	db_scooter_notifications
analytics-service	8086	Eventos de arriendo y estadísticas de usuario/sistema.	db_scooter_analytics
logistics-service	8087	Ubicación GPS de patinetas y búsqueda por área.	db_scooter_logistics
maintenance-service	8088	Issues de mantención de patinetas (reporte y ciclo de vida).	db_scooter_maintenance
support-service	8089	Tickets de soporte al cliente (asignación y resolución).	db_scooter_support
station-service	8090	Estaciones de carga: capacidad, dock/undock, búsqueda por cercanía.	db_scooter_stations
review-service	8091	Reseñas 1–5 por arriendo y rating promedio por patineta.	db_scooter_reviews

El BFF no cuenta como microservicio de dominio: la plataforma posee **10 microservicios funcionales** más el Gateway y el BFF.

2.3. Diagramas Arquitectónicos (C4 Model)

Diagrama C2 (Contenedores)

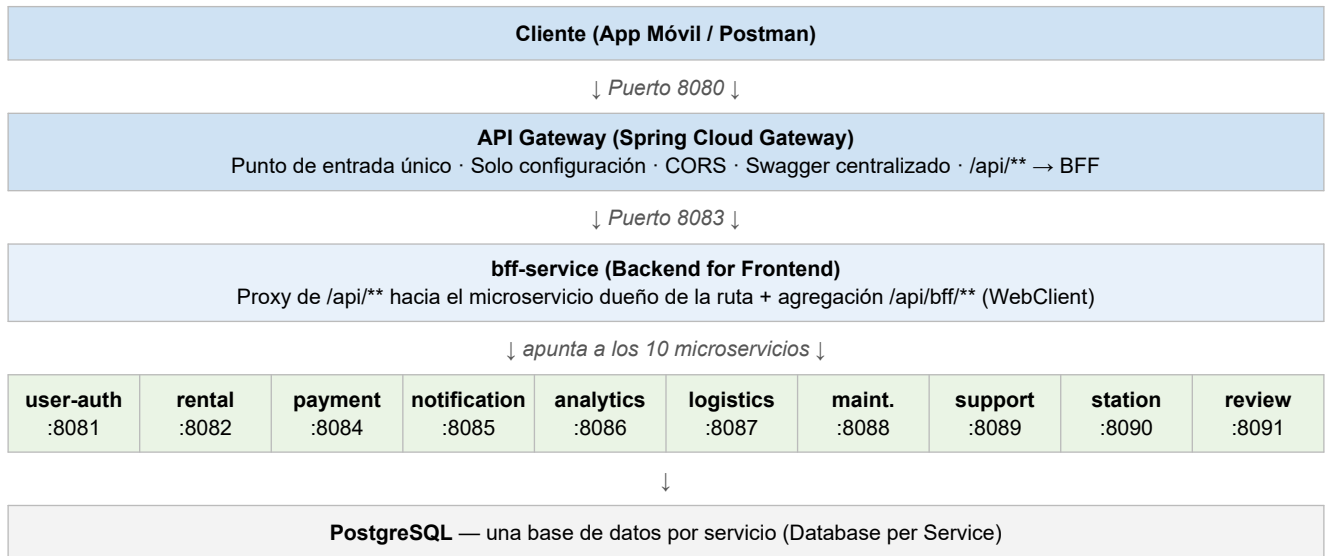
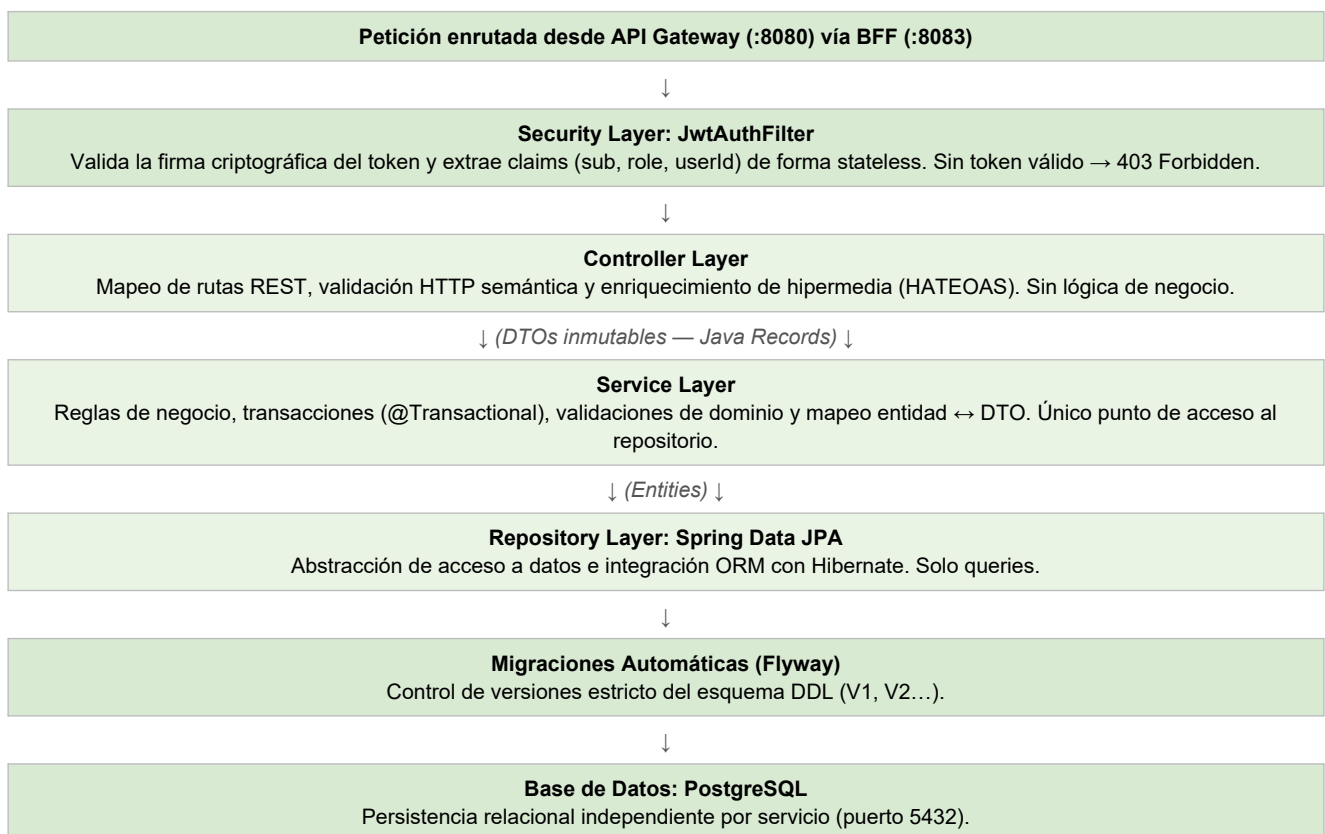


Diagrama C3 (Componentes — patrón CSR)

Todos los microservicios siguen el patrón **CSR (Controller → Service → Repository)**: las capas se comunican intercambiando **DTOs inmutables (Java Records)**, ningún controller accede a un repositorio directamente y la **capa Client** (presente en el bff-service) es la responsable de la comunicación externa hacia los demás microservicios.



2.4. Estándares de Desarrollo, Clean Code y Pruebas

El repositorio ha sido codificado íntegramente en inglés, aplicando el principio de Responsabilidad Única (SRP) para asegurar que los controladores permanezcan libres de lógica, delegando la carga transaccional a la capa de servicios. Las directrices aplicadas incluyen:

- **Patrón CSR verificado:** Los 10 microservicios implementan las tres capas controller/service/repository; se verificó que ningún controller inyecta repositorios directamente.
- **Control Global de Excepciones:** Utilización de `@RestControllerAdvice` con manejadores específicos (`@ExceptionHandler`) para interceptar excepciones de negocio (404 Not Found, 400 Bad Request, 401 Unauthorized) y serializarlas en un formato JSON coherente y predecible.
- **Flyway y Datos de Prueba:** Separación absoluta entre la definición estructural de la base de datos (DDL) mediante Flyway y la hidratación de datos (DML). Se integró la biblioteca DataFaker para poblar escenarios realistas exclusivamente bajo el perfil dev, garantizando despliegues limpios en producción.
- **Estrategia Integral de Pruebas:** Suite de testing jerárquica con pruebas unitarias de servicios (Mockito aislado), pruebas de controladores (MockMvc) y pruebas de integración contextuales (`@SpringBootTest`) — 11 clases de test que incluyen los nuevos `StationServiceTest` y `ReviewServiceTest`. El entorno de pruebas aísla la infraestructura con una base H2 en memoria configurada mediante `application-test.yml`.
- **Optimización de Flujo:** Adopción sistemática de la API de Streams de Java para transformar colecciones de manera funcional y declarativa, minimizando bucles imperativos propensos a errores.

2.5. Documentación de API (Swagger centralizado)

La documentación OpenAPI está **centralizada en el API Gateway**: un único Swagger UI disponible en <http://localhost:8080/swagger-ui.html> agrupa, mediante un selector de definiciones, la documentación de los 10 microservicios y del BFF. El Gateway expone cada `/v3/api-docs` bajo rutas `/docs/{servicio}` implementadas únicamente con configuración (filtro `SetPath`), preservando el principio de que el Gateway solo enruta. Adicionalmente, cada servicio mantiene su propio Swagger UI (puertos 8081–8091), y la colección Postman incluida en el repositorio cubre todos los endpoints, incluyendo las carpetas de estaciones, reseñas, BFF y acceso vía Gateway.

2.6. Estrategia de Control de Versiones (Git)

El repositorio se gestiona en GitHub aplicando un flujo de trabajo basado en ramas de funcionalidad (*Feature Branch Workflow*), que mantiene la rama principal siempre estable y desplegable:

- **main:** Rama estable y siempre desplegable. Contiene la versión integrada y probada de los 12 servicios (API Gateway, BFF y los 10 microservicios de dominio).
- **feature/*:** Una rama por funcionalidad o microservicio. El desarrollo se realiza de forma aislada y se integra a main mediante Pull Requests, lo que permite revisión de código antes del merge.
- **Ramas de soporte (docs/, test/):** Empleadas para documentación, la colección Postman y la incorporación de pruebas, manteniendo separadas las tareas que no modifican la lógica de negocio.

Los commits siguen una convención semántica de mensajes (feat:, test:, docs:, merge:) que facilita la trazabilidad del historial. Las integraciones que generan conflictos se resuelven mediante merges explícitos y documentados, preservando un historial limpio y comprensible.

3. Conclusión

El diseño y construcción de la plataforma VeloUrbe evidencia la aplicación rigurosa de metodologías avanzadas de ingeniería de software. La segregación funcional en **10 microservicios**, coordinada a través de un **Gateway reactivo sin código** que delega en un **BFF** como orquestador y proxy único hacia los servicios, asegura un sistema altamente cohesivo, tolerante a fallos y preparado para soportar fluctuaciones de concurrencia en tiempo real.

La integración de Java 21, combinada con prácticas no negociables como el patrón CSR verificado en todas las capas, la inmutabilidad de transferencia de datos, el versionamiento estricto de esquemas con Flyway, la documentación centralizada con Swagger y una cobertura de pruebas automatizadas, certifica el cumplimiento absoluto de los requerimientos técnicos y de calidad exigidos. VeloUrbe se posiciona, por tanto, como una solución arquitectónica sólida, mantenible y estratégicamente diseñada para la evolución continua en el sector de

la movilidad urbana.